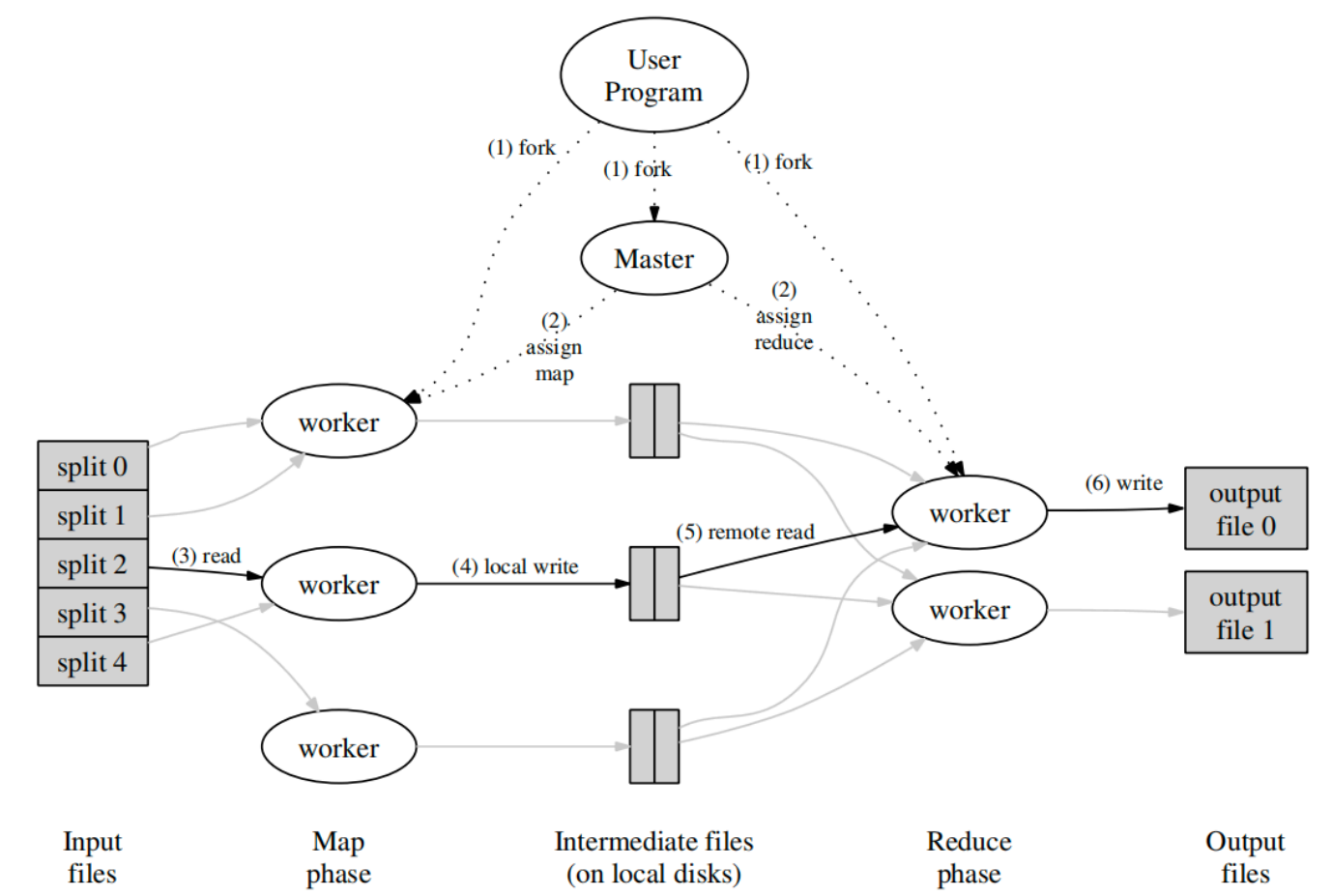


MapReduce

MapReduce的执行流程

【论文阅读】 MapReduce: Simplified Data Processing on Large Clusters-CSDN博客



分布式计算系统，主要解决如何在TB级别的数据上进行大量计算。Google希望能够在数以千计的计算机上执行TB级别的计算。

具体而言，设计设计一种分布式框架，能够组织大量的计算机，负责分发和移动数据，能够处理故障等。工程师只需要实现应用程序的核心，就能够见应用程序运行在数以千计的计算机上，分布式框架实现如何将远算分发到多台计算机上。

MapReduce的核心思想是：应用程序设计人员和分布式运算的使用者，只需要写简单的Map函数和Reduce函数，而不需要知道任何有关分布式的事情，MapReduce框架会处理剩下的事情。

#####

流程：

- 输入会被分割成M份文件；
- 对于每一份文件，MapReduce 会调用 `Map()` ，并且产生 `<key, value>` 列表；
- 当所有 `Map` 执行完后，对于每种 `key` ，MapReduce 会调用 `Reduce()` 收集所有的中间结果 `value` ；
- `Reduce()` 最终会返回 `<key, value>` 集合作为最终的输出结果。

```
Input1 -> Map -> a,1 b,1
Input2 -> Map ->      b,1
Input3 -> Map -> a,1      c,1
                |   |   |
                |   |   -> Reduce -> c,1
                |   -----> Reduce -> b,2
                -----> Reduce -> a,2
```

单词频率统计案例：

```
Map(k, v)
    split v into words
    for each word w
        emit(w, "1")
Reduce(k, v_list)
    emit(len(v_list))
```

实现细节：

- Server发送 `map/reduce` 代码给worker
- Server需要追踪 `map/reduce` 任务的进度
- 将 `Map()` 的中间结果打乱，作为 `Reduce()` 的输入
- 在各个worker之间进行 **负载均衡**
- **容错**，从崩溃的Worker节点中进行恢复。

可扩展性分析：

- N 倍的 worker 结点可以获得 N 倍吞吐量。
- Map/Reduce 任务都是并行执行，且worker结点之间不存在交互。
- 越多的worker节点，越多的吞吐量。

容错机制

Coordinator 故障

- MapReduce中只有单个的Coordinator，单个结点发生故障的概率往往很低。Coordinator会周期性地将其数据存储 **checkpoint**，如果Coordinator的任务掉线(dies)，它能够从最近的checkpoint中重新启动。

Map worker 崩溃：当Coordinator 发现 worker 不在响应 ping时，worker被认为是发生崩溃。

- Coordinator 是知道哪些Map任务被分配给崩溃的Worker。
- 如果这些Map任务的中间结果丢失，需要再其他正常的worker上重新执行这些任务。
- 如果这些Map任务的中间结果已近被Reduce获取了，无需重新执行这些任务。

Reduce worker 崩溃

- Coordinator在其他正常的Worker上重新启动没有完成的Reduce任务。

Coordinator 给两个Worker节点相同的 `Reduce()` 任务，他们都尝试在GFS上向相同的结果文件写入数据

- GFS的原子命名会阻止这样的事情，最后只有一个文件可以被看到。

负载均衡策略

落单者(Straggler)：如果 $N - 1$ 个 worker 等待 1 个 **非常慢** 的 worker 节点完成工作，这使得整个 MapReduce Job 非常慢。但是，有些 worker 上的任务可能会花费比其他 worker 更多的时间。

一种解决方法：当任务的数量多于 worker 的数量时，Coordinator 为提前完成任务的 worker 分配新的任务，快的 worker 会比慢的 worker 执行更多的任务，使得所有的 worker 在尽可能相同的时间完成任务（该方法的前提是没有特别长的任务占据整个 Map/Reduce 任务）。

MapReduce 的应用单词计数

```
package main

//
// a word-count application "plugin" for MapReduce.
//
// go build -buildmode=plugin wc.go
//

import "6.5840/mr"
import "unicode"
import "strings"
import "strconv"

// The map function is called once for each file of input. The first
// argument is the name of the input file, and the second is the
// file's complete contents. You should ignore the input file name,
// and look only at the contents argument. The return value is a slice
// of key/value pairs.
func Map(filename string, contents string) []mr.KeyValue {
    // function to detect word separators.
    ff := func(r rune) bool { return !unicode.IsLetter(r) }

    // split contents into an array of words.
```

```

words := strings.FieldsFunc(contents, ff)

kva := []mr.KeyValue{}
for _, w := range words {
    kv := mr.KeyValue{w, "1"}
    kva = append(kva, kv)
}
return kva
}

// The reduce function is called once for each key generated by the
// map tasks, with a list of all the values created for that key by
// any map task.
func Reduce(key string, values []string) string {
    // return the number of occurrences of this word.
    return strconv.Itoa(len(values))
}

```

MapReduce 顺序实现

```

package main

//
// simple sequential MapReduce.
//
// go run mrsequential.go wc.so pg*.txt
//

import "fmt"
import "6.5840/mr"
import "plugin"
import "os"
import "log"
import "io/ioutil"
import "sort"

// for sorting by key.
type ByKey []mr.KeyValue

```

```

// for sorting by key.
func (a ByKey) Len() int           { return len(a) }
func (a ByKey) Swap(i, j int)      { a[i], a[j] = a[j], a[i] }
func (a ByKey) Less(i, j int) bool { return a[i].Key < a[j].Key }

func main() {
    if len(os.Args) < 3 {
        fmt.Fprintf(os.Stderr, "Usage: mrsequential xxx.so
inputfiles...\n")
        os.Exit(1)
    }

    mapf, reducef := loadPlugin(os.Args[1])

    //
    // read each input file,
    // pass it to Map,
    // accumulate the intermediate Map output.
    //
    intermediate := []mr.KeyValue{}
    for _, filename := range os.Args[2:] {
        file, err := os.Open(filename)
        if err != nil {
            log.Fatalf("cannot open %v", filename)
        }
        content, err := ioutil.ReadAll(file)
        if err != nil {
            log.Fatalf("cannot read %v", filename)
        }
        file.Close()
        kva := mapf(filename, string(content))
        intermediate = append(intermediate, kva...)
    }

    //
    // a big difference from real MapReduce is that all the
    // intermediate data is in one place, intermediate[],
    // rather than being partitioned into NxM buckets.

```

```

//

sort.Sort(ByKey(intermediate))

oname := "mr-out-0"
ofile, _ := os.Create(oname)

//
// call Reduce on each distinct key in intermediate[],
// and print the result to mr-out-0.
//
i := 0
for i < len(intermediate) {
    j := i + 1
    for j < len(intermediate) && intermediate[j].Key ==
intermediate[i].Key {
        j++
    }
    values := []string{}
    for k := i; k < j; k++ {
        values = append(values, intermediate[k].Value)
    }
    output := reducef(intermediate[i].Key, values)

    // this is the correct format for each line of Reduce output.
    fmt.Fprintf(ofile, "%v %v\n", intermediate[i].Key, output)

    i = j
}

ofile.Close()
}

// load the application Map and Reduce functions
// from a plugin file, e.g. ../mrapps/wc.so
func loadPlugin(filename string) (func(string, string) []mr.KeyValue,
func(string, []string) string) {
    p, err := plugin.Open(filename)
    if err != nil {

```

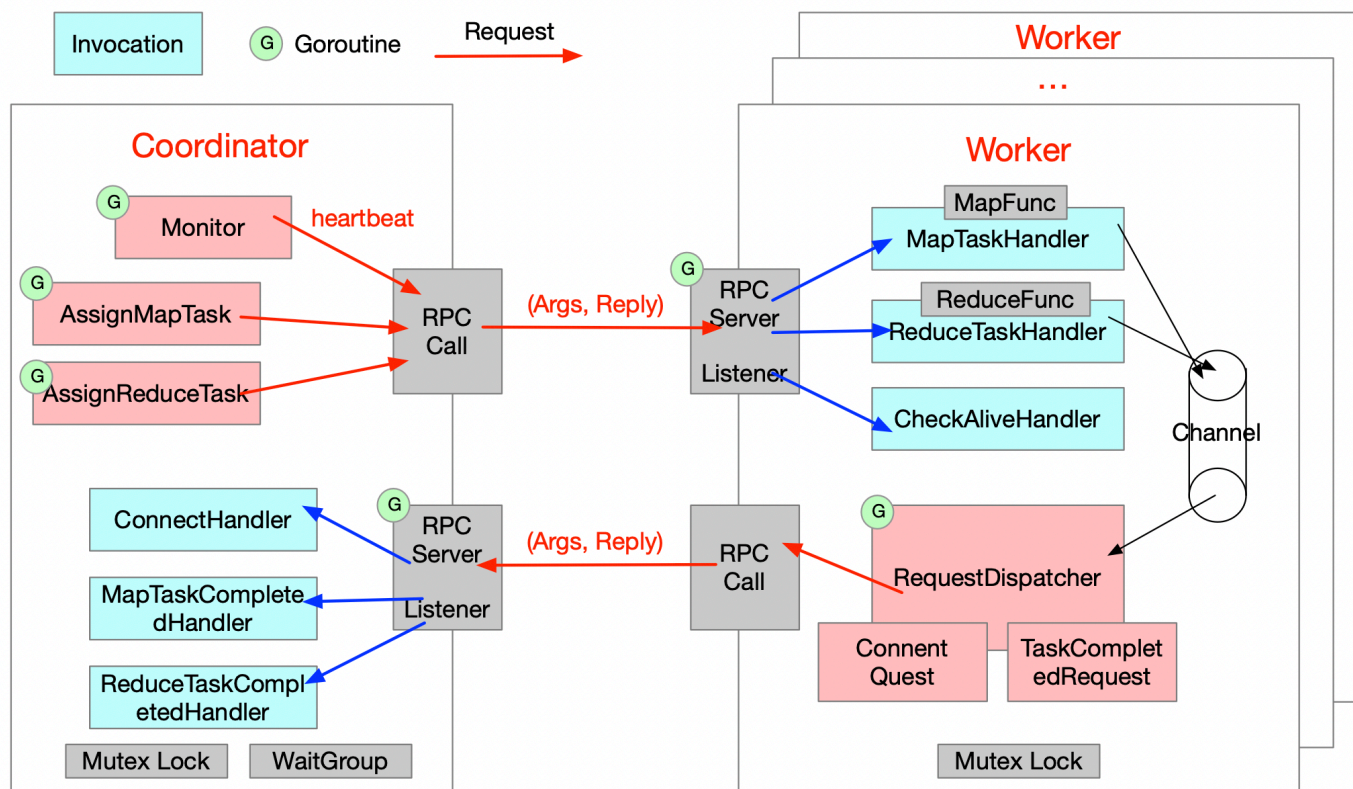
```

    log.Fatalf("cannot load plugin %v", filename)
}
xmapf, err := p.Lookup("Map")
if err != nil {
    log.Fatalf("cannot find Map in %v", filename)
}
mapf := xmapf.(func(string, string) []mr.KeyValue)
xreducef, err := p.Lookup("Reduce")
if err != nil {
    log.Fatalf("cannot find Reduce in %v", filename)
}
reducef := xreducef.(func(string, []string) string)

return mapf, reducef
}

```

MapReduce 分布式实现



顶层接口设计

在 `main/mrcoordinator.go` 包中实现了启用一个 `Coordinator` 的代码，而 `Coordinator` 代码具体逻辑的实现在 `mr/coordinator.go`

```
package main

//
// start the coordinator process, which is implemented
// in ../mr/coordinator.go
//
// go run mrcoordinator.go pg*.txt
//
// Please do not change this file.
//

import "6.5840/mr"
import "time"
import "os"
import "fmt"

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintf(os.Stderr, "Usage: mrcoordinator inputfiles...\n")
        os.Exit(1)
    }

    m := mr.MakeCoordinator(os.Args[1:], 5)
    for m.Done() == false {
        time.Sleep(time.Second)
    }

    time.Sleep(time.Second)
}
```

在 `main/mrworker.go` 中实现了启用一个 `Worker` 进程的方法，而 `Worker` 的具体的逻辑在 `mr/worker.go` 中

```

package main

//
// start a worker process, which is implemented
// in ../mr/worker.go. typically there will be
// multiple worker processes, talking to one coordinator.
//
// go run mrworker.go wc.so
//
// Please do not change this file.
//

import "6.5840/mr"
import "plugin"
import "os"
import "fmt"
import "log"

func main() {
    if len(os.Args) != 2 {
        fmt.Fprintf(os.Stderr, "Usage: mrworker xxx.so\n")
        os.Exit(1)
    }

    mapf, reducef := loadPlugin(os.Args[1])

    mr.Worker(mapf, reducef)
}

// load the application Map and Reduce functions
// from a plugin file, e.g. ../mrapps/wc.so
func loadPlugin(filename string) (func(string, string) []mr.KeyValue,
func(string, []string) string) {
    p, err := plugin.Open(filename)
    if err != nil {
        log.Fatalf("cannot load plugin %v", filename)
    }
    xmapf, err := p.Lookup("Map")
    if err != nil {

```

```

    log.Fatalf("cannot find Map in %v", filename)
}
mapf := xmapf.(func(string, string) []mr.KeyValue)
xreducef, err := p.Lookup("Reduce")
if err != nil {
    log.Fatalf("cannot find Reduce in %v", filename)
}
reducef := xreducef.(func(string, []string) string)

return mapf, reducef
}

```

Master 和 Woker RPC通信

Coordinator 和 Woker 通过 Go Rpc 实现通信, 结构体 Args 和 Reply 定义了一次RPC请求的请求参数和响应的参数类型

```

//
// example to show how to declare the arguments
// and reply for an RPC.
//

type Args struct {
    Request string
    Data map[string]string
}

type Reply struct {
    Data map[string]string
}

```

下面的常量定义了, 在 Coordinator 和 Worker 发RPC请求时, 所有请求的类型:

```
const (
    REQUEST_CONNECT = "Connect"
    REQUEST_MAP     = "Map"
    REQUEST_REDUCE  = "Reduce"
    REQUEST_INTERMEDIATE = "Intermediate"
    REQUEST_COMPLETED_JOB = "Completed_Job"
    REQUEST_COMPLETED_MAP_TASK = "Completed_Map"
    REQUEST_COMPLETED_REDUCE_TASK = "Completed_Reduce"
    REQUEST_CHECK_WORKER_ALIVE = "Check_Worker_Alive"
)
```

在编程实现过程中，服务器端需要注册结构体对象，然后通过对象所属的方法暴露给调用者，从而提供服务，该方法称之为输出方法，此输出方法可以被远程调用。当然，在定义输出方法时，能够被远程调用的方法需要遵循一定的规则。我们通过代码进行讲解：

```
func (t *T) MethodName(request T1,response *T2) error
```

- 对外暴露的方法有且只能有两个参数，这个两个参数只能是输出（可导出）类型或内建类型，两种类型中的一种。
- 方法的第二个参数必须是指针类型。
- 方法的返回类型为error。
- 方法的类型是可输出（可导出）的。
- 方法本身也是可输出（可导出）的。

通过接口 interface 定义 Coordinator 和 Worker 能够为外界提供服务的接口：

```
type MapReduceCoordinatorService interface {
    // coordinator
    ReceiveConnectHandler(args *Args, reply *Reply) error
    MapTaskCompletedHandler(args *Args, reply *Reply) error
    ReduceTaskCompletedHandler(args *Args, reply *Reply) error
}

type MapReduceWorkerService interface {
```

```
// worker
MapTaskHandler(args *Args, reply *Reply) error
ReduceTaskHandler(args *Args, reply *Reply) error
ReturnIntermediateHandler(args *Args, reply *Reply) error
ReceiveJobFinishedHandler(args *Args, reply *Reply) error
ReceiveCheckAliveHandler(args *Args, reply *Reply) error
}
```

服务器端需要注册结构体对象，然后通过对象所属的方法暴露给调用者。其中，`class` 和 `instance` 分别定义了注册结构体的类型和实例对象。`mode` 指定以 `http` 还是 `tcp` 建立连接，`address` 指定了监听的端口。

```
func PRCServer(mode string, address string, class string, instance
interface{}) {
    rpc.RegisterName(class, instance) // register
    if listener, err := net.Listen(mode, address); err != nil { // get
the listener
        log.Fatal("ListenTCP error:", err)
    } else {
        go func(){ // start a thread to listen
            for {
                if conn, err := listener.Accept(); err != nil { // block
                    log.Fatal("Accept error:", err)
                } else {
                    go rpc.ServeConn(conn) // start a thread to deal with the
request
                }
            }
        }()
    }
}
```

Go RPC 远程过程调用提供了 **同步调用** 和 **异步调用** 两种方式。客户端使用了同步的调用方式，即一直等待服务端的响应或出错。在等待的过程中，客户端就不能处理其它的任务了。其中，`is_sync=true` 表示启用同步调用，否则启用异步调用。

```
func RPCCall(class string, method string, args interface{},
```

```

        reply interface{}, address string, is_sync bool) bool {
    client, err := rpc.Dial("tcp", address)
    // defer client.Close()
    if err != nil {
        log.Printf("The device[%s] may be dead. Dial error:%s\n", address,
err)
        return false
    }
    rpcname := fmt.Sprintf("%s.%s", class, method)
    if is_sync {
        err = client.Call(rpcname, args, reply) // sync
        if err != nil {
            log.Printf("The device[%s] may be dead. Call error:%s", address,
err)
            return false
        }
    } else {
        client.Go(rpcname, args, reply, nil) // async
    }
    return true
}

```

Coordinator 设计与实现

Coordinator的数据结构

- `files []string`: 字符串切片类型，存储所有需要处理的文件，每个文件被视为一个任务Map任务
- `workers []string`: 字符串切片类型，持有所有与服务器 `Coordinator` 建立连接的worker的身份标识(ip 地址)
- `workerSatate map[string]string`: map映射类型，字符串类型的键表示worker主机的地址，值表示worker的状态。`idle` 表示worker 处于空闲状态，`in-progress` 表示worker正在执行任务，`completed` 表示任务已经结束。

- `location` : map映射类型，其中key是worker主机的地址，Map任务执行结果在该主机存放的位置
- `workerTaskList` : map映射类型，记录了每个worker执行任务及其任务状态
- `nMap` : `Map` 任务的数量
- `nReduce` : `Reduce` 任务的数量
- `mux sync.Mutex` : 上述切片和映射等数据结构都是线程不安全的，在并发环境下，通过对共享数据加显式锁，实现Go程地序列化访问。
- `wgForMapTask` : 当Worker节点处于就绪状态时且满足最低可用数量时，用于触发Map任务控制器分发Map任务
- `wgForReduceTask` : 当所有的Map任务都完成时，用于通知Reduce任务控制器分发Reduce任务
- `wgForCompletion` : 当所有的Reduce任务都完成时，用于关闭 `Coordinator` 服务

```
type Coordinator struct {
    files []string // slice of filename, taskid-filename
    workers []string // workerid, address(the identity of the worker)
    workerState map[string]string // {address: state(idle, in-progress, completed)}
    location map[string]string // {address: location}
    workerTaskList map[string]map[string]bool // {address: {map/reduce-taskid: true/false}}

    // There are M map tasks and R reduce tasks to assign
    nMap int // the number of map
    nReduce int // the number of reduce
    minWorker int

    mux sync.Mutex // lock
    wgForMapTask sync.WaitGroup
    wgForReduceTask sync.WaitGroup
    wgForCompletion sync.WaitGroup
}
```

`Coordinator` 构造器，在构造完成`Coordinator`对象之后，启动

`c.netSocketServer()` RPC服务端监听程序，监听来自客户端的RPC请求。

```
//
// create a Coordinator.
// main/mrcoordinator.go calls this function.
// nReduce is the number of reduce tasks to use.
//
func MakeCoordinator(files []string, nReduce int) *Coordinator {
    c := Coordinator{ // create a Coordinator
        files:files,
        workers:make([]string, 0),
        location:make(map[string]string),
        workerState:make(map[string]string),
        workerTaskList:make(map[string]map[string]bool),
        nMap: NUM_MAP, nReduce:nReduce, minWorker: MIN_NUM_WORKER,
    }
    c.wgForMapTask.Add(MIN_NUM_WORKER)
    c.wgForReduceTask.Add(NUM_MAP)
    c.wgForCompletion.Add(nReduce)
    c.netSocketServer() // start tcp server
    return &c
}
```

RPC Server 监听器和Worker监控器

启动 Coordinator 服务。首先，Coordinator会启动RPC Server服务器，创建一个goroutine阻塞式监听来Worker的请求，针对每一个来Worker的请求都创建一个独立的goroutine处理该请求。然后，Coordinator会在后台启动一个监控器 `monitor`，该监控器采用心跳包机制监控所有连接到的Worker的健康状态，每隔 2s 会向Worker发送心跳包。

接着，Coordinator 会启动Map任务分发器和Reduce任务分发器。

```
func (c *Coordinator) netSocketServer() {
    PRCServer("tcp", ":1234", "Coordinator", c)
    c.monitor() // monitor on backgroubd
}
```

```

    go func(c *Coordinator) { c.assignMapTask()(c) } // assignMapTask is
executed only once
    go func(c *Coordinator) { c.assignReduceTask()(c)
}

func (c *Coordinator) monitor() {
    go func(c *Coordinator) {
        for {
            time.Sleep(2000 * time.Millisecond) // sleep 1000ms
            c.checkWorkerAlive()
        }
    }(c)
}

```

心跳包机制，Coordinator向所有的Worker发生一次请求 REQUEST_CHECK_WORKER_ALIVE，该请求不携带任何数据。如果Worker节点响应，则认为该Worker节点状态正常。

```

func (c *Coordinator) checkWorkerAlive() {
    c.mux.Lock()
    backup := utils.Copy(c.workerState).(map[string]string)
    c.mux.Unlock()
    for addr, state := range backup {
        if state == STATE_FAILED { continue }
        args := Args {
            Request: REQUEST_CHECK_WORKER_ALIVE,
            Data: nil,
        }
        if !RPCCall(CLIENT, RECEIVE_CHECK_ALIVE_HANDLER, &args, &Reply{},
addr, false) {
            c.mux.Lock()
            c.workerState[addr] = STATE_FAILED
            c.mux.Unlock()
        }
    }
}

```

`ReceiveConnectHandler` 负责处理Worker建立连接的请求，将Worker节点加入Coordinator注册表 `workers` 中，初始化该节点的状态 `workerState` 。

```
func (c *Coordinator) ReceiveConnectHandler(args *Args, reply *Reply)
error {
    address := args.Data["ip"] + ":" + args.Data["port"]
    c.mux.Lock()
    if _, state := c.workerState[address]; !state { // ignore repeated
request
        c.workers = append(c.workers, address)
        c.workerState[address] = STATE_IDLE // init state with idle
        if len(c.workers) <= c.minWorker {
            defer c.wgForMapTask.Done()
        } else {
            c.minWorker = -1
        }
        log.Println("Worker[" + address + "] requests to establish
connection.")
    }
    c.mux.Unlock()
    return nil
}
```

Map 任务分发器

- Map任务控制器在至少在有 `MIN_NUM_WORKER` 个Worker成功连接到 `Coordinator` 后才会分发Map任务给Worker节点。
- 对于所有待分配的任务，Map任务控制器会轮训一次所有的Worker，找到第一个可以的Worker机器，将Map任务分配给该机器
- 请求的类型是 `REQUEST_MAP`，请求的数据包括文件路径 `filepath`、文件名 `filename`、`mapTaskId` Map任务Id、Reduce任务的数量 `nReduce`
- Map任务控制器会更新该Worker机器的状态，初始化Map任务该Worker机器任务列表中状态

```

// Assign Alogorithm.
// The coordinator picks idle workers and assigns each one a map
task.
// At least one worker survived.
// A worker may execute multiple map tasks.
// 'return false' means that all workers are dead.
func (c *Coordinator) assignMapTask() bool {
    c.wgForMapTask.Wait()
    // time.Sleep(5000*time.Millisecond)
    dir, _ := os.Getwd() // get the dir
    c.mux.Lock()
    defer c.mux.Unlock()
    for taskid, filename := range c.files {
        args := Args {
            Request: REQUEST_MAP,
            Data: map[string]string {
                "filepath": path.Join(dir, filename), "filename": filename,
                "mapTaskId": strconv.Itoa(taskid), "nReduce":
strconv.Itoa(c.nReduce),
            },
        }
        step, id := 0, taskid % len(c.workers)
        for c.workerState[c.workers[id]] == STATE_FAILED ||
            !RPCCall(CLIENT, MAP_TASK_HANDLER, &args, &Reply{},
c.workers[id], false) { // faul-tolerant
            step, id = step + 1, id + 1
            if id >= len(c.workers) { id = 0 }
            if step >= len(c.workers) { return false }
        }
        addr := c.workers[id]
        c.workerState[addr] = STATE_IN_PROGRESS // update the state
        if c.workerTaskList[addr] == nil { c.workerTaskList[addr] =
make(map[string]bool) }
        c.workerTaskList[addr][fmt.Sprintf("map-%d", taskid)] = false //
record the map{worker, taskid}
    }
    log.Println("[Server] assigns map task.")
    return true // destroy goroutine
}

```

MapTaskCompletedHandler 处理来自Map Worker的请求，更新Map Worker完成Map任务的状态。如果该Worker已经完成了所有的Map任务，则更新Map Worker的状态为空闲 STATE_IDLE

```
func (c *Coordinator) MapTaskCompletedHandler(args *Args, reply
*Reply) error {
    address := args.Data["ip"] + ":" + args.Data["port"]
    c.mux.Lock()
    defer c.mux.Unlock()
    if c.workerTaskList[address][fmt.Sprintf("map-%s",
args.Data["taskId"])] { // ignore repeated requests
        log.Printf("Worker[%s] send repeated request!\n", address)
        return nil
    }
    c.workerTaskList[address][fmt.Sprintf("map-%s",
args.Data["taskId"])] = true // update task state
    idle := true
    for taskid, state := range c.workerTaskList[address] {
        if strings.Contains(taskid, "map") && !state {
            idle = false
            break
        }
    }
    if idle { c.workerState[address] = STATE_IDLE } // update the
worker's state
    c.location[address] = args.Data["fileLocation"] // save the
intermediate location from worker
    log.Printf("Worker[%s] has completed map task whose taskid is %s.
\n", address, args.Data["taskId"])
    defer c.wgForReduceTask.Done()
    return nil
}
```

Reduce 任务分发器

- 当所有的Map任务都完成后，Reduce任务控制器被唤醒，将 `nReduce` 个Reduce任务依次分发给处于可用状态的 `Worker` 节点。
- Reduce任务控制器将所有中间结果存的位置（服务器ip, 文件路径）序列化字符串后 `json.Marshal(c.location)`，以请求的数据形式发送给执行Reduce任务的节点
- 请求的类型 `REQUEST_REDUCE`，请求的数据包括Reduce任务id `reduceTaskId`，所有中间结果存放位置 `location`
- 最后，Reduce会更新这些执行Reduce任务的worker节点的状态，以及初始化对应任务列表中任务的状态

```
//  
// The coordinator picks idle workers and assigns each one a reduce  
task  
//  
func (c *Coordinator) assignReduceTask() bool {  
    c.wgForReduceTask.Wait()  
    c.mux.Lock()  
    defer c.mux.Unlock()  
    str, err := json.Marshal(c.location)  
    if err != nil { log.Fatal(err)}  
    for taskid := 0; taskid < c.nReduce; taskid ++ {  
        args := Args {  
            Request: REQUEST_REDUCE,  
            Data: map[string]string {  
                "reduceTaskId": strconv.Itoa(taskid), "location": string(str),  
            },  
        }  
        step, id := 0, taskid % len(c.workers)  
        for c.workerState[c.workers[id]] == STATE_FAILED ||  
            !RPCCall(CLIENT, REDUCE_TASK_HANDLER, &args, &Reply{},  
c.workers[id], false) { // faul-tolerant  
            step, id = step + 1, id + 1  
            if id >= len(c.workers) { id = 0 }  
            if step >= len(c.workers) { return false }  
        }  
        addr := c.workers[id]
```

```

    c.workerState[addr] = STATE_IN_PROGRESS // update worker state
    if c.workerTaskList[addr] == nil { c.workerTaskList[addr] =
make(map[string]bool) }
    c.workerTaskList[addr][fmt.Sprintf("reduce-%d", taskid)] = false
// record the map{worker, taskid}
    log.Printf("[Server] assigns worker[%s]-task[%d]\n",
c.workers[id],taskid)
}
log.Println("[Server] assigns reduce task.")
return true// destroy goroutine
}

```

`MapTaskCompletedHandler` 处理来自Reduce Worker的请求，更新Reduce Worker完成Reduce任务的状态。如果该Worker已经完成了所有的Reduce任务，则更新Reduce Worker的状态为空闲 `STATE_IDLE`

将Reduce任务的结果 `args.Data["output"]` 保存到本地

```

func (c *Coordinator) ReduceTaskCompletedHandler(args *Args, reply
*Reply) error {
    address := args.Data["ip"] + ":" + args.Data["port"]
    c.mux.Lock()
    if c.workerTaskList[address][fmt.Sprintf("reduce-%s",
args.Data["taskId"])] {
        log.Printf("Worker[%s] send repeated request!\n", address) //
ignore repeated requests
        c.mux.Unlock()
        return nil
    }
    c.workerTaskList[address][fmt.Sprintf("reduce-%s",
args.Data["taskId"])] = true // update task state
    completed := true
    for taskid, state := range c.workerTaskList[address] {
        if strings.Contains(taskid, "reduce") && !state {
            completed = false
            break
        }
    }
}

```

```

    if completed { c.workerState[address] = STATE_COMPLETED } // update
the worker's state
    c.mux.Unlock()
    output := utils.MapUnmarshal(args.Data["output"]) // deserialize
    utils.SaveMapToFile(".", "mr-out-" + args.Data["taskId"], ".txt",
output) // save the output
    log.Printf("Worker[%s] has completed reduce task whose taskid is
%s.\n", address, args.Data["taskId"])
    // c.wgForCompletion.Done() // If some things cause the goroutine to
fail without causing the program to fail, c.wgForReduceTask.Done will
never be executed. So the parent-goroutine will wait forever.
    defer c.wgForCompletion.Done()
    return nil
}

```

Worker 设计与实现

`KeyValue` 用于封装MapFunc的计算结果，MapFunc的计算结果用 `KeyValue` 的列表表示

```

type KeyValue struct {
    Key    string
    Value  string
}

```

Worker的数据结构

- `mapf` 是用户提供的函数，用于实现完成Map任务
- `reducef` 是用户提供的函数，用于完成Reduce任务
- `ip`：用于表示该worker计算程序所属机器的地址
- `port`：用于标识worker计算程序的访问端口
- `mapTaskList`：需要处理的Map任务列表，每一个元素是map任务id

- `reduceTaskId`：需要处理的Reduce任务列表，每一个元素是reduce任务id

```
type Client struct {
    mapf func(string, string) []KeyValue
    reducef func(string, []string) string
    ip string
    port string
    mapTaskId []string // list of map-task-id
    reduceTaskId []string // list of reduce-task-id
    channel chan map[string]interface{}
    mux sync.Mutex
}
```

启动一个Worker服务，实例化Client对象，启动RPC Server服务器，监听来自Coordinator的请求

```
//
// main/mrworker.go calls this function.
//
func Worker(mapf func(string, string) []KeyValue,
    reducef func(string, []string) string) {
    port, err := utils.GetFreePort()
    if err != nil {
        log.Fatal("Can't get free port. Error:", err)
        return
    }
    client := Client{
        mapf:mapf, reducef:reducef,
        ip:"localhost", port: strconv.Itoa(port),
        mapTaskId:make([]string, 0), reduceTaskId:make([]string, 0),
        channel: make(chan map[string]interface{}),
    }
    PRCServer("tcp", ":" + strconv.Itoa(port), "Client", &client) //
listen server
    if ok := client.requestConnect(); !ok { // connect server
        return
    }
}
```

```

log.Println("Client[" + client.ip + ":" + client.port + "]" + "
connects to coordinator successfully and waits for task from
coordinator.")
// go client.consumer()
client.dispatcher() // dispatch request
}

```

在Worker实例化后，Worker会向Coordinator发送建立连接的请求，将Worker注册到Coordinator中：

- 请求类型是 `REQUEST_CONNECT`
- 请求的参数包括Worker的 `ip` 和端口 `port`，该端口是Worker监听的端口

```

func (c *Client) requestConnect() bool {
    args := Args {
        Request: REQUEST_CONNECT,
        Data: map[string]string{
            "ip": c.ip,
            "port": c.port,
        },
    }
    return RPCCall(COORDINATOR, RECEIVE_CONNECT_HANDLER, &args,
        &Reply{},
        SERVER_IP + ":" + SERVER_PORT, false)
}

```

Map任务处理器 `MapTaskHandler`

- 从请求的数据 `args.Data` 中获取文件的位置，读取文件的内容，调用用户定义的 `MapFunc` 执行Map任务
- 将 `MapFunc` 返回的计算结果（`KeyValue` 的列表）按 `Key` 划分成 `nReduce` 份，每一份数据都是一个Reduce任务的输入
- 将划分后的Map任务计算结果持久化到硬盘上，每一份结果存储到以 `mr-mapTaskId-reduceTaskId` 格式命名的文本文件中

- 通过 Channel 向 RequestDispatcher 发送「Map任务完成」的请求 REQUEST_COMPLETED_MAP_TASK。

```
// A worker who is assigned a map task reads the contents of the
// contents of the corresponding input split.
// execute map task
func (c *Client) MapTaskHandler(args *Args, reply *Reply) error {
    log.Printf("[%s:%s]Receive the map task!\n", c.ip, c.port)
    // time.Sleep(5000 * time.Millisecond)
    if content, err := utils.ReadFromTxt(args.Data["filepath"]); err ==
nil {
        kva := c.mapf(args.Data["filename"], content) // execute the map
operation and the intermediate key/value pairs are buffered in memory
        if nReduce, err := strconv.Atoi(args.Data["nReduce"]); err == nil
{ // split kva into c.NReduce pieces
            partitions := partition(kva, nReduce)
            for i, part := range(partitions) {
                filename := "mr-" + args.Data["mapTaskId"] + "-" +
strconv.Itoa(i) + JSON_POSTFIX
                intermediate(filename, part.kva) // the buffered pairs are
written to local disk,periodically
            }
        } else {
            log.Fatal("Can't transform string into int. Error:", err)
        }
    } else {
        log.Fatalf("Can't read %v", args.Data["filename"])
    }
    c.mux.Lock()
    c.mapTaskId = append(c.mapTaskId, args.Data["mapTaskId"])
    c.mux.Unlock()
    c.channel <- map[string]interface{}{
        "request":REQUEST_COMPLETED_MAP_TASK, "taskid":
args.Data["mapTaskId"],
    }
    return nil
}
```

划分算法 `Partition`。对Map任务的计算结果 `[]KeyValue` 按Key的字典序从小到大，将Key的哈希码相同的 `KeyValue` 分配到同一Reduce任务中。

```
// use ihash(key) % NReduce to choose the reduce
// task number for each KeyValue emitted by Map.
func ihash(key string) int {
    h := fnv.New32a()
    h.Write([]byte(key))
    return int(h.Sum32() & 0x7fffffff)
}

// split kva into c.NReduce pieces
func partition(kva []KeyValue, NReduce int) []Partition {
    backup := make([]KeyValue, len(kva))
    copy(backup, kva) // get the copy
    sort.Slice(backup, func (i int, j int) bool { // sort by Key
        return backup[i].Key < backup[j].Key
    })
    partitions := make([]Partition, NReduce)
    for _, kv := range backup {
        pos := ihash(kv.Key) % NReduce
        if partitions[pos].kva == nil {
            partitions[pos].kva = make([]KeyValue, 0)
        }
        partitions[pos].kva = append(partitions[pos].kva, kv)
    }
    return partitions
}
```

Map任务计算结果持久化到本地磁盘中

```
// save the map result as intermediate such as map-X-Y
func intermediate(filename string, kva []KeyValue) {
    dir, _ := os.Getwd()
    filePath := path.Join(dir, "../", INTERMEDIATE, filename)
    file, err := os.Create(filePath)
    defer file.Close()
    if err != nil {
        log.Fatalf("Can't create the %v. Error: %v", filename, err)
    }
}
```

```

    return
}
enc := json.NewEncoder(file)
for _, kv := range kva { // kv is a copy
    if err := enc.Encode(&kv); err != nil {
        log.Fatalf("Can't write. Error: %v", err)
        break
    }
}
}
}

```

请求发送器 RequestDispatcher

RequestDispatcher 负责调度发送给 Coordinator

```

func (c *Client) dispatcher() {
    for {
        value := <- c.channel
        if value["request"] == REQUEST_COMPLETED_MAP_TASK {
            c.requestCompleteMapTask(MAP_TASK, value["taskid"].(string))
            log.Println("Complete map task! Wait for new task from
coordinator.")
        } else if value["request"] == REQUEST_COMPLETED_REDUCE_TASK {
            c.requestCompleteReduceTask(REDUCE_TASK, value["taskid"].
(string),
                                value["res"].(map[string]string))
            log.Println("Complete reduce task! Wait for new task from
coordinator.")
        } else if value["request"] == REQUEST_COMPLETED_JOB {
            log.Println("MapReduce job is Finished!")
            return
        }
    }
}
}

```

Worker发送请求通知Coordinator, 其负责的Map任务已经计算完成:

- 请求类型是 `REQUEST_COMPLETED_MAP_TASK`
- 请求的参数包括Worker的 `ip` 和端口 `port`（用于标识是哪个Worker完成了Map任务），任务类型 `taskType`、任务ID `taskId` 和Map计算结果持久化到本地的文件路径 `filePath`

```
func (c *Client) requestCompleteMapTask(taskType string, taskId
string) bool {
    dir, _ := os.Getwd()
    filePath := path.Join(dir, "../", INTERMEDIATE)
    args := Args {
        Request: REQUEST_COMPLETED_MAP_TASK,
        Data: map[string]string{
            "ip": c.ip,
            "port": c.port,
            "taskType": taskType,
            "taskId": taskId,
            "fileLocation": filePath,
        },
    }
    return RPCCall(COORDINATOR, MAP_TASK_COMPLETED_HANDLER, &args,
&Reply{},
        SERVER_IP + ":" + SERVER_PORT, false)
}
```

Worker发送请求通知Coordinator，其负责的Reduce任务已经计算完成：

- 请求的类型是 `REQUEST_COMPLETED_REDUCE_TASK`
- 请求的参数包括Worker的 `ip` 和端口 `port`（用于标识是哪个Worker完成了Map任务），任务类型 `taskType`、任务ID `taskId` 和Reduce计算结果 `output`，其被序列化成字符串

```
func (c *Client) requestCompleteReduceTask(taskType string, taskId
string, res map[string]string) bool {
    args := Args {
        Request: REQUEST_COMPLETED_REDUCE_TASK,
        Data: map[string]string {
            "ip": c.ip,
```

```

        "port": c.port,
        "taskType": taskType,
        "taskId": taskid,
        "output": utils.Marshal(res),
    },
}
return RPCCall(COORDINATOR, REDUCE_TASK_COMPLETED_HANDLER, &args,
&Reply{},
    SERVER_IP + ":" + SERVER_PORT, false)
}

```

Reduce 任务处理器

- Worker 收到 Coordinator 的调用通知来完成Reduce任务
- Worker能够从Coordinator的请求数据中解析出Reduce任务的 Id `reduceTaskId`，以及该Reduce任务需要处理的数据所在的位置 `location` `map[string]string`。其中 `location` 的 `key` 是待处理数据所在主机的 `address`，`value` 是数据在该主机上的文件路径
- Worker 给持有Reduce任务 `reduceTaskId` 的主机发送消息获取输入数据 `[]KeyValue`
- 对所有的 `[]KeyValue` 按Key的字典序从小到大排序
- 调用用户提供 `ReduceFunc` 执行计算Reduce任务
- 通过RequestDispatcher组件向Coordinator发送「Reduce任务完成」的请求 `REQUEST_COMPLETED_REDUCE_TASK`，请求中携带Reduce任务的计算结果 `map[string]string`

```

// When a reduce worker is notified by the coordinator about the
locations,
// it uses RPC to read the buffered data from the local disk of the
map workers.
// execute reduce task.

```

```

func (c *Client) ReduceTaskHandler(args *Args, reply *Reply) error {
    log.Printf("[%s:%s]Receive the reduce task!\n", c.ip, c.port)
    // time.Sleep(5000 * time.Millisecond)
    reduceTaskId := args.Data["reduceTaskId"]
    location := utils.MapUnmarshal(args.Data["location"]) // deserialize
    kva := c.requestIntermediate(reduceTaskId, location) // get the
    inermedidate from other workers(file server)
    sort.Slice(kva, func (i int, j int) bool { // sort kva by Key so
    that the same key is grouped together
        return kva[i].Key < kva[j].Key
    })
    res := make(map[string]string) // save the reduce output
    for i := 0; i < len(kva); {
        j := i + 1
        for j < len(kva) && kva[j].Key == kva[i].Key {
            j ++
        }
        values := []string{}
        for k := i; k < j; k ++ {
            values = append(values, kva[k].Value)
        }
        res[kva[i].Key] = c.reducef(kva[i].Key, values)
        i = j
    }
    c.mux.Lock()
    c.reduceTaskId = append(c.reduceTaskId, reduceTaskId)
    c.mux.Unlock()
    c.channel <- map[string]interface{} {
        "request":REQUEST_COMPLETED_REDUCE_TASK, "taskid":reduceTaskId,
"res":res,
    }
    return nil
}

```

Reduce Worker向持有Reduce输入数据的Map Worker节点发送RPC请求:

- 针对多所有执行Map任务的Map Worker主机发送获取 Intermediate 数据的请求

- 请求类型是 `REQUEST_INTERMEDIATE` , 请求体数据包括待执行的Reduce任务id `reduceTaskId` 和文件路径 `location`
- Map Worker响应的数据 `reply.Data["intermediate"]` 可能为空。将响应数据进行反序列化得到 `var temp []KeyValue` 。
- 汇总id为 `reduceTaskId` Reduce任务的输入数据 `kva` ,

```
func (c *Client) requestIntermediate(reduceTaskId string, location
map[string]string) []KeyValue {
    kva := make([]KeyValue, 0)
    for addr, filePath := range location {
        args := Args {
            Request: REQUEST_INTERMEDIATE,
            Data: map[string]string {
                "reduceTaskId": reduceTaskId,
                "location": filePath,
            },
        }
        reply := Reply {}
        RPCCall(CLIENT, RETURN_INTERMEDIATE_HANDLER, &args, &reply, addr,
true)
        var temp []KeyValue
        if err := json.Unmarshal([]byte(reply.Data["intermediate"]),
&temp); err != nil {
            log.Fatal("Unmarshal failed. Error", err)
        }
        kva = append(kva, temp...)
    }
    return kva
}
```

Map Worker响应请求

- Map Worker可能完成多个Map任务，因此会产生多个Map任务计算结果。Map Worker查找其计算过的所有的Map任务id `maptaskid` , 结合请求参数中携带的Reduce任务id `reduceTaskId` , 得到Map任务计算结果存储的文件名字
- 依次将本地磁盘文件中的计算结果读取到内存中，存储到 `[]KeyValue` 缓存中

- 为了在网络中传输数据，将 []KeyValue 序列化成字符串，作为响应的数据 reply.Data

```
func (c *Client) ReturnIntermediateHandler(args *Args, reply *Reply)
error {
    kva := make([]KeyValue, 0)
    for _, maptaskid := range c.mapTaskId {
        filename := "mr-" + maptaskid + "-" + args.Data["reduceTaskId"] +
JSON_POSTFIX
        temp := dejsonize(path.Join(args.Data["location"], filename)) //
read data([] KeyValue) from json file
        kva = append(kva, temp...)
    }
    if str, err := json.Marshal(kva); err != nil {
        log.Fatal("Serialization failed! Error:", err)
        return nil
    } else {
        reply.Data = map[string]string { "intermediate":string(str), }
    }
    log.Printf("Receive task of reading intermediate
file(reduceTaskId=%s)!\n", args.Data["reduceTaskId"])
    return nil
}
```

Map Worker从持久化到本地的json文件中读取数据

```
func dejsonize(filePath string) []KeyValue {
    file, err := os.Open(filePath)
    defer file.Close()
    if err != nil {
        log.Fatal("Can't open file. Error:", err)
        return nil
    }
    kva := make([]KeyValue, 0)
    dec := json.NewDecoder(file)
    for {
        var kv KeyValue
        if err := dec.Decode(&kv); err != nil {
```

```
        break
    }
    kva = append(kva, kv)
}
return kva
}
```